

# MovieLens-1M数据集详解与 推荐系统面试问题总结

## 目录

|          |                          |          |
|----------|--------------------------|----------|
| <b>1</b> | <b>MovieLens-1M数据集概述</b> | <b>2</b> |
| 1.1      | 数据集基本信息                  | 2        |
| 1.2      | 原始数据文件详解                 | 2        |
| 1.2.1    | movies.dat - 电影信息        | 2        |
| 1.2.2    | users.dat - 用户信息         | 3        |
| 1.2.3    | ratings.dat - 评分记录       | 3        |
| 1.3      | 数据预处理与划分                 | 4        |
| 1.3.1    | 冷启动阈值设定                  | 4        |
| 1.3.2    | 数据集划分                    | 4        |
| <b>2</b> | <b>特征工程详解</b>            | <b>5</b> |
| 2.1      | 特征编码方案                   | 5        |
| 2.1.1    | One-hot特征（6个）            | 5        |
| 2.1.2    | Multi-hot特征（2个）          | 5        |
| 2.2      | 特征空间计算                   | 6        |
| 2.3      | 数据格式说明                   | 6        |
| 2.3.1    | 基础格式（17列）                | 6        |
| 2.3.2    | 带历史信息格式（197列）            | 6        |
| <b>3</b> | <b>推荐系统面试问题详解</b>        | <b>7</b> |
| 3.1      | 数据集相关问题                  | 7        |
| 3.1.1    | Q1: 介绍下数据集               | 7        |
| 3.1.2    | Q2: 哪些特征做了哪些编码           | 7        |
| 3.1.3    | Q3: 电影名字怎么编码，是字典化的吗      | 7        |
| 3.1.4    | Q4: 字典多少个                | 7        |
| 3.1.5    | Q5: 年龄怎么编号               | 7        |

|       |                                           |    |
|-------|-------------------------------------------|----|
| 3.2   | 模型架构问题 . . . . .                          | 7  |
| 3.2.1 | Q6: 为什么对embedding做梯度下降而不是对生成器参数 . . . . . | 7  |
| 3.2.2 | Q7: batch_a和batch_b是什么关系 . . . . .        | 8  |
| 3.2.3 | Q8: 冷启动的本质是什么 . . . . .                   | 8  |
| 3.2.4 | Q9: 为什么需要cold_loss和warm_loss . . . . .    | 8  |
| 3.2.5 | Q10: 预测时怎么做 . . . . .                     | 8  |
| 3.3   | 技术细节问题 . . . . .                          | 8  |
| 3.3.1 | Q11: W_SENET参数是什么 . . . . .               | 8  |
| 3.3.2 | Q12: 去噪模块怎么实现 . . . . .                   | 8  |
| 3.3.3 | Q13: 对比学习怎么做 . . . . .                    | 9  |
| 3.3.4 | Q14: GAUC怎么实现 . . . . .                   | 9  |
| 3.3.5 | Q15: 512阈值怎么来的 . . . . .                  | 9  |
| 3.4   | 工程实现问题 . . . . .                          | 9  |
| 3.4.1 | Q16: 技术栈是什么 . . . . .                     | 9  |
| 3.4.2 | Q17: 训练时间多久 . . . . .                     | 9  |
| 3.4.3 | Q18: 为什么和别人项目一样 . . . . .                 | 9  |
| 3.4.4 | Q19: 如何处理batch内的多条数据 . . . . .            | 10 |
| 3.4.5 | Q20: 新item信息不确定怎么办 . . . . .              | 10 |
| 3.5   | 理论理解问题 . . . . .                          | 10 |
| 3.5.1 | Q21: MAML和本项目的区别 . . . . .                | 10 |
| 3.5.2 | Q22: 为什么除以20而不是21 . . . . .               | 10 |
| 3.5.3 | Q23: Scaling Law对推荐系统的影响 . . . . .        | 10 |
| 3.5.4 | Q24: 为什么用历史用户去噪 . . . . .                 | 10 |
| 3.5.5 | Q25: 预测时没有梯度下降机会怎么办 . . . . .             | 11 |
| 4     | 总结                                        | 11 |

# 1 MovieLens-1M数据集概述

## 1.1 数据集基本信息

MovieLens-1M是明尼苏达大学GroupLens研究组发布的经典推荐系统数据集。

规模：

- 评分数量：1,000,209条
- 用户数量：6,040个
- 电影数量：3,706部
- 评分范围：1-5分（整数）
- 时间跨度：2000年左右

数据密度：

- 平均每个用户评分：165.6条
- 平均每部电影被评分：269.9次
- 稀疏度：95.5%（大部分用户-电影对没有交互）

## 1.2 原始数据文件详解

数据集包含三个核心文件，均采用双冒号(::)分隔：

### 1.2.1 movies.dat - 电影信息

格式：MovieID::Title::Genres

示例：

```
1 1::Toy Story (1995)::Animation|Children's|Comedy
2 2::Jumanji (1995)::Adventure|Children's|Fantasy
```

字段说明：

- MovieID：电影唯一标识，范围1-3952
- Title：电影名称（英文），包含上映年份
- Genres：电影类型，用竖线(—)分隔，共18种类型

**18种电影类型：** Action, Adventure, Animation, Children's, Comedy, Crime, Documentary, Drama, Fantasy, Film-Noir, Horror, Musical, Mystery, Romance, Sci-Fi, Thriller, War, Western

### 1.2.2 users.dat - 用户信息

格式: UserID::Gender::Age::Occupation::Zip-code

示例:

```
1 1::F::1::10::48067
2 2::M::56::16::70072
```

字段说明:

- UserID: 用户唯一标识, 范围1-6040
- Gender: 性别, M(男)/F(女)
- Age: 年龄段编码 (7个档位)
- Occupation: 职业编码 (21种职业)
- Zip-code: 邮政编码 (本项目未使用)

年龄段编码:

- 1: "Under 18"
- 18: "18-24"
- 25: "25-34"
- 35: "35-44"
- 45: "45-49"
- 50: "50-55"
- 56: "56+"

职业编码 (部分): 0-其他, 1-学术/教育, 2-艺术家, 3-文员, 4-大学生, 7-工程师, 10-律师, 12-程序员, 16-退休, 17-销售, 20-作家等

### 1.2.3 ratings.dat - 评分记录

格式: UserID::MovieID::Rating::Timestamp

示例:

```
1 1::1193::5::978300760
2 1::661::3::978302109
```

字段说明:

- UserID: 用户ID
- MovieID: 电影ID
- Rating: 评分, 1-5分 (整数)
- Timestamp: Unix时间戳 (本项目未使用)

## 1.3 数据预处理与划分

### 1.3.1 冷启动阈值设定

**512阈值规则:**

- 交互次数  $\geq 512$ : 老item (90%)
- 交互次数  $\leq 512$ : 新item (10%)
- 阈值选择: 按9:1比例划分时, 恰好卡在512条交互

**统计结果:**

- 老item: 约3,335部电影 (90%)
- 新item: 约371部电影 (10%)
- 新item平均交互数: 约100-200条

### 1.3.2 数据集划分

**训练集:**

- big\_train\_main.csv: 老item的所有交互数据
- 用于预训练DNN基础模型
- 约90万条评分记录

**元学习集:**

- train\_oneshot\_a\_w\_ngb\_hist.csv: 新item的batch\_a数据
- train\_oneshot\_b\_w\_ngb\_hist.csv: 新item的batch\_b数据
- 同一个item的不同交互记录, 用于MAML训练
- 每个约5万条记录

测试集：

- test\_onehot\_a\_hist.csv: 新item的支持集
- test\_onehot\_b\_hist.csv: 新item的查询集
- 用于评估冷启动性能

## 2 特征工程详解

### 2.1 特征编码方案

项目采用混合编码方式，包含6个one-hot特征和2个multi-hot特征。

#### 2.1.1 One-hot特征（6个）

1. **user\_id**: 用户ID, 6040个取值
2. **movie\_id**: 电影ID, 3706个取值
3. **gender**: 性别, 2个取值 (M/F)
4. **age**: 年龄段, 7个取值 (1,18,25,35,45,50,56)
5. **occupation**: 职业, 21个取值
6. **release\_year**: 上映年份, 约80个取值 (1920-2000)

#### 2.1.2 Multi-hot特征（2个）

1. **title**: 电影标题词汇
  - 最多5个词（不足补0）
  - 词典大小: 约3000个英文单词
  - 编码方式: 词频统计后分配ID
2. **genres**: 电影类型
  - 最多5个类型（不足补0）
  - 类型数量: 18种
  - 编码方式: 类型名称映射到ID

## 2.2 特征空间计算

总特征维度:  $n_{ft} = 11134$

计算方式:

$$\begin{aligned} n_{ft} &= \text{user\_id} + \text{movie\_id} + \text{gender} + \text{age} \\ &\quad + \text{occupation} + \text{release\_year} + \text{title\_vocab} + \text{genres} \\ &= 6040 + 3706 + 2 + 7 + 21 + 80 + 3000 + 18 \\ &= 11134 \end{aligned}$$

## 2.3 数据格式说明

### 2.3.1 基础格式 (17列)

示例:

|   |   |      |     |   |   |      |       |      |      |      |      |      |    |   |   |   |   |
|---|---|------|-----|---|---|------|-------|------|------|------|------|------|----|---|---|---|---|
| 1 | 1 | 6909 | 195 | 9 | 3 | 2217 | 11087 | 4726 | 3232 | 2235 | 3233 | 3234 | 10 | 0 | 0 | 0 | 0 |
|---|---|------|-----|---|---|------|-------|------|------|------|------|------|----|---|---|---|---|

列含义:

- 第1列: label (评分, 0/1二分类)
- 第2-7列: 6个one-hot特征 (user\_id, movie\_id, gender, age, occupation, release\_year)
- 第8-17列: 10个multi-hot特征 (5个title词 + 5个genres类型)

### 2.3.2 带历史信息格式 (197列)

示例:

|   |      |      |      |      |      |      |       |      |      |      |      |      |    |   |   |   |   |
|---|------|------|------|------|------|------|-------|------|------|------|------|------|----|---|---|---|---|
| 1 | 1    | 6909 | 195  | 9    | 3    | 2217 | 11087 | 4726 | 3232 | 2235 | 3233 | 3234 | 10 | 0 | 0 | 0 | 0 |
| 2 | 5359 | 5381 | 5413 | 5453 | 5517 | 5518 | 5520  | 5558 | 5570 | 5627 | 5651 | 5664 |    |   |   |   |   |
| 3 | 5703 | 5724 | 5727 | 5777 | 5790 | 5796 | 5818  | 5845 | ...  |      |      |      |    |   |   |   |   |

列含义:

- 第1-17列: 基础特征 (同上)
- 第18-177列: 160列邻居特征 (20个邻居 × 8个特征)
- 第178-197列: 20个历史用户ID (用于去噪)

历史用户ID (hist特征):

- 含义: 最近点击过当前电影的20个用户ID

- 作用：通过用户embedding平均来增强item embedding
- 去噪原理：多个用户的平均表示更稳定，减少噪声影响

## 3 推荐系统面试问题详解

### 3.1 数据集相关问题

#### 3.1.1 Q1: 介绍下数据集

回答要点：

MovieLens-1M包含100万条评分，6040个用户，3706部电影。我们将交互次数 $\leq 512$ 的定义为老item（90%）， $> 512$ 的为新item（10%），用于模拟冷启动场景。数据包含用户属性（性别、年龄、职业）和电影属性（标题、类型、年份），共8个特征维度。

#### 3.1.2 Q2: 哪些特征做了哪些编码

回答要点：

6个one-hot特征：user\_id、movie\_id、gender、age、occupation、release\_year。2个multi-hot特征：title（最多5个词）、genres（最多5个类型）。总特征空间11134维。

#### 3.1.3 Q3: 电影名字怎么编码，是字典化的吗

回答要点：

是的，电影标题经过分词后建立词典。统计所有电影标题的词频，高频词分配较小ID。每个标题取前5个词，不足补0。词典大小约3000个英文单词。

#### 3.1.4 Q4: 字典多少个

回答要点：

英文词典约3000个单词。MovieLens-1M是英文数据集，不涉及中文。

#### 3.1.5 Q5: 年龄怎么编号

回答要点：

7个年龄段：1(Under 18)、18(18-24)、25(25-34)、35(35-44)、45(45-49)、50(50-55)、56(56+)。直接使用数据集提供的编码，无需额外处理。

### 3.2 模型架构问题

#### 3.2.1 Q6: 为什么对embedding做梯度下降而不是对生成器参数

回答要点：



推荐系统的目标是快速适应新item，而不是学习新任务的模型参数。对embedding做梯度下降可以直接优化item表示，适应新交互数据。生成器参数在元学习阶段已经学好，预测时保持不变。

### 3.2.2 Q7: batch\_a和batch\_b是什么关系

回答要点：

同一个item的不同交互记录。batch\_a用于第一次梯度下降（模拟few-shot场景），batch\_b用于评估适应后的效果。这样生成器学会如何生成易于快速适应的embedding。

### 3.2.3 Q8: 冷启动的本质是什么

回答要点：

新item没有历史交互，无法训练出有效的ID embedding。通过属性特征（标题、类型、年份）生成初始embedding，再利用少量交互数据快速调整，实现快速冷启动。

### 3.2.4 Q9: 为什么需要cold\_loss和warm\_loss

回答要点：

标准MAML只优化warm\_loss（适应后的性能）。推荐系统需要同时考虑zero-shot（直接用生成器）和few-shot（加warm-up）场景，所以用 $0.1 \times \text{cold\_loss} + 0.9 \times \text{warm\_loss}$ 联合优化。

### 3.2.5 Q10: 预测时怎么做

回答要点：

支持两种模式：1) Zero-shot：直接用生成器输出的embedding预测；2) Few-shot：收集少量交互数据（如10-50条），在batch\_a上做一次梯度下降，得到适应后的embedding再预测。

## 3.3 技术细节问题

### 3.3.1 Q11: W\_SENET参数是什么

回答要点：

$3 \times 3$ 的注意力权重矩阵，用于自适应调整3个属性特征（release\_year、title、genres）的重要性。采用SENet思想，通过特征的全局信息动态计算权重。

### 3.3.2 Q12: 去噪模块怎么实现

回答要点：

用最近点击该电影的20个用户的embedding平均来增强item embedding。具体流程：生成器输出→ 拼接20个用户embedding → 平均池化→ MLP → 最终embedding。除以20而非21，给用户信息更高权重。

### 3.3.3 Q13: 对比学习怎么做

回答要点：

简化版对比学习，batch级别整体相似度。同一item的不同batch（batch\_a和batch\_b）作为正样本对，最大化它们的相似度。没有显式的item级负样本采样。

### 3.3.4 Q14: GAUC怎么实现

回答要点：

代码实现了GAUC但实际用的是普通AUC。GAUC是按user\_id分组，每个用户单独计算AUC后加权平均，权重是该用户的样本数。更适合评估个性化推荐效果。

### 3.3.5 Q15: 512阈值怎么来的

回答要点：

按9:1比例划分老item和新item时，统计所有电影的交互次数，排序后90%分位点恰好在512条。这是数据驱动的阈值，不是人为设定。

## 3.4 工程实现问题

### 3.4.1 Q16: 技术栈是什么

回答要点：

Python 3.6 + PyTorch 1.7 + NumPy 1.19。数据格式用TFRecord（兼容性考虑）。训练用Adam优化器，学习率1e-3。Embedding维度10，batch size 256。

### 3.4.2 Q17: 训练时间多久

回答要点：

GPU（RTX 3090）上3-5分钟。预训练DNN 1-2分钟（老item数据），元学习训练2-3分钟（新item数据）。CPU上约20-30分钟。

### 3.4.3 Q18: 为什么和别人项目一样

回答要点：

基于ICML 2019论文复现，但我深入理解了每个技术细节。比如为什么对embedding而非参数做梯度下降、batch\_a和batch\_b的关系、去噪模块的设计原理等。面试更看重技术理解深度而非原创性。

### 3.4.4 Q19: 如何处理batch内的多条数据

#### 回答要点:

batch内256条数据可能包含多个不同item。对每个item，用其属性特征生成embedding，然后与对应的用户embedding拼接，输入DNN预测CTR。batch训练提高效率，每个样本独立计算loss后平均。

### 3.4.5 Q20: 新item信息不确定怎么办

#### 回答要点:

Multi-hot特征支持变长输入。Title和genres都padding到固定长度（5个slot），不足补0。生成器对padding位置的embedding权重会自动学习为接近0，不影响最终表示。

## 3.5 理论理解问题

### 3.5.1 Q21: MAML和本项目的区别

#### 回答要点:

标准MAML学习模型参数的初始化，使其能快速适应新任务。本项目学习embedding生成器，使其输出的embedding易于快速适应。本质都是meta-learning，但优化对象不同。

### 3.5.2 Q22: 为什么除以20而不是21

#### 回答要点:

设计选择，给用户信息更高权重（95%）。如果除以21，相当于用户和item初始embedding权重相同。除以20是一种残差连接思想，保留更多用户协同信号。

### 3.5.3 Q23: Scaling Law对推荐系统的影响

#### 回答要点:

模型性能与数据量、参数量呈幂律关系。推荐系统中冷启动问题更严重，因为新item数据量极少，无法充分利用大模型能力。元学习通过迁移老item知识，缓解数据稀缺问题。

### 3.5.4 Q24: 为什么用历史用户去噪

#### 回答要点:

新item的初始embedding由属性生成，可能不准确。历史交互用户的embedding包含真实的协同过滤信号。通过平均多个用户embedding，可以降低单个用户的噪声，得到更稳定的item表示。

### 3.5.5 Q25: 预测时没有梯度下降机会怎么办

回答要点：

支持两种模式。Zero-shot直接用生成器输出预测，适合完全新item。Few-shot收集少量交互（10-50条）作为支持集，做一次梯度下降后预测，效果更好但需要等待数据积累。

## 4 总结

本项目基于MovieLens-1M数据集，实现了基于元学习的推荐系统冷启动方案。核心创新点：

1. 用属性特征生成初始embedding，解决新item无历史数据问题
2. 元学习训练生成器，使embedding易于快速适应
3. 历史用户embedding去噪，提升冷启动效果
4. 支持zero-shot和few-shot两种预测模式

技术栈：Python + PyTorch，训练时间3-5分钟（GPU），适合工业落地。